Sprint -4,5,6,7

SPRINT 4 — AI AGENTS + WORKFLOW RECORDER

Features 4 and 19

PREREQUISITE: Sprints 1-3 complete

PASTE THIS INTO REPLIT AI AGENT

FEATURE 4: AI AUTONOMOUS AGENTS

tRPC Router: `src/server/routers/agents.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import { agentQueue } from "@/jobs/queue-config";

export const agentsRouter = router({
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      return prisma.aiAgent.findMany({
        where: {
          workspaceId: input.workspaceId,
          deletedAt: null,
          OR: [
            { visibility: "workspace" },
```

```
            { createdBy: ctx.user.id },
            { visibleToUserIds: { has: ctx.user.id } },
          ],
        },
        orderBy: { createdAt: "desc" },
      });
    }),

  getById: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.aiAgent.findUnique({
        where: { id: input.id },
        include: { runs: { take: 10, orderBy: { createdAt: "d
esc" } } },
      });
    }),

  create: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      name: z.string().min(1).max(100),
      description: z.string().optional(),
      icon: z.string().optional(),
      systemPrompt: z.string().min(10),
      model: z.string().default("gpt-4o"),
      temperature: z.number().min(0).max(2).default(0.7),
      capabilities: z.array(z.string()),
      dataAccess: z.object({
        allPages: z.boolean().default(false),
        specificPageIds: z.array(z.string()).default([]),
        allDatabases: z.boolean().default(false),
        specificDatabaseIds: z.array(z.string()).default([]),
        allChannels: z.boolean().default(false),
        specificChannelIds: z.array(z.string()).default([]),
        allMeetings: z.boolean().default(false),
```

```
      }),
      triggerType: z.enum(["manual", "scheduled", "event", "m
essage_in_channel", "webhook"]).default("manual"),
      triggerConfig: z.any().optional(),
      visibility: z.enum(["private", "workspace", "specific_m
embers"]).default("workspace"),
      visibleToUserIds: z.array(z.string().uuid()).optional
(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.aiAgent.create({
        data: { ...input, createdBy: ctx.user.id, type: "cust
om" },
      });
    }),

  update: protectedProcedure
    .input(z.object({
      id: z.string().uuid(),
      name: z.string().optional(),
      description: z.string().optional(),
      icon: z.string().optional(),
      systemPrompt: z.string().optional(),
      model: z.string().optional(),
      temperature: z.number().optional(),
      capabilities: z.array(z.string()).optional(),
      dataAccess: z.any().optional(),
      triggerType: z.string().optional(),
      triggerConfig: z.any().optional(),
      visibility: z.string().optional(),
      visibleToUserIds: z.array(z.string()).optional(),
      isActive: z.boolean().optional(),
    }))
    .mutation(async ({ input }) => {
      const { id, ...data } = input;
      return prisma.aiAgent.update({ where: { id }, data });
```

```
    }),

  delete: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.aiAgent.update({ where: { id: input.id },
data: { deletedAt: new Date() } });
    }),

  // Chat with an agent
  chat: protectedProcedure
    .input(z.object({
      agentId: z.string().uuid(),
      message: z.string().min(1).max(5000),
      conversationId: z.string().uuid().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      const agent = await prisma.aiAgent.findUnique({ where:
{ id: input.agentId } });
      if (!agent) throw new Error("Agent not found");

      // Get or create conversation
      let conversation;
      if (input.conversationId) {
        conversation = await prisma.agentConversation.findUni
que({ where: { id: input.conversationId } });
      }
      if (!conversation) {
        conversation = await prisma.agentConversation.create
({
          data: {
            agentId: input.agentId,
            userId: ctx.user.id,
            messages: [{ role: "user", content: input.messag
e, timestamp: new Date().toISOString() }],
          },
```

```
        });
      } else {
        const messages = conversation.messages as any[];
        messages.push({ role: "user", content: input.message,
timestamp: new Date().toISOString() });
        await prisma.agentConversation.update({
          where: { id: conversation.id },
          data: { messages },
        });
      }

      // Execute agent (enqueue for background processing)
      const run = await prisma.agentRun.create({
        data: {
          agentId: agent.id,
          workspaceId: agent.workspaceId,
          triggeredBy: "manual",
          triggerData: { message: input.message, conversation
Id: conversation.id },
          invokedByUserId: ctx.user.id,
          status: "queued",
          steps: [],
        },
      });

      // For manual chat, execute inline (not background) for
instant response
      // Build messages for AI
      const conversationMessages = (conversation.messages as
any[]).map((m: any) => ({
        role: m.role === "user" ? "user" as const : "assistan
t" as const,
        content: m.content,
      }));

      // Build tools based on agent capabilities
```

```
      const tools = buildAgentTools(agent.capabilities);

      const aiMessages = [
        { role: "system" as const, content: agent.systemPromp
t },
        ...conversationMessages,
      ];

      const response = await aiService.generateWithTools(aiMe
ssages, tools, {
        model: agent.model as any,
        temperature: agent.temperature,
      });

      // Process tool calls if any
      let finalResponse = response.content || "";
      const steps: any[] = [];

      if (response.tool_calls && response.tool_calls.length >
0) {
        for (const toolCall of response.tool_calls) {
          const stepResult = await executeAgentTool(
            toolCall.function.name,
            JSON.parse(toolCall.function.arguments),
            agent,
            ctx.user.id,
          );
          steps.push({
            stepNumber: steps.length + 1,
            action: toolCall.function.name,
            input: JSON.parse(toolCall.function.arguments),
            output: stepResult,
            status: "success",
          });
        }
```

```
        // Get final response after tool execution
        const followUp = await aiService.generateText(
          `Based on these tool results, provide your final re
sponse:\n${JSON.stringify(steps.map(s => ({ action: s.action,
result: s.output })))}`,
          { systemPrompt: agent.systemPrompt, model: agent.mo
del as any },
        );
        finalResponse = followUp.text;
      }

      // Save agent response to conversation
      const updatedMessages = [...(conversation.messages as a
ny[])];
      updatedMessages.push({ role: "agent", content: finalRes
ponse, timestamp: new Date().toISOString(), runId: run.id });
      await prisma.agentConversation.update({
        where: { id: conversation.id },
        data: { messages: updatedMessages },
      });

      // Update run
      await prisma.agentRun.update({
        where: { id: run.id },
        data: { status: "completed", finalOutput: finalRespon
se, steps, completedAt: new Date() },
      });

      return {
        conversationId: conversation.id,
        response: finalResponse,
        runId: run.id,
        steps,
      };
    }),
```

```
  // Get conversation history
  getConversation: protectedProcedure
    .input(z.object({ agentId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      return prisma.agentConversation.findFirst({
        where: { agentId: input.agentId, userId: ctx.user.id,
status: "active" },
        orderBy: { updatedAt: "desc" },
      });
    }),

  // Get run history
  getRuns: protectedProcedure
    .input(z.object({ agentId: z.string().uuid(), limit: z.nu
mber().default(20) }))
    .query(async ({ input }) => {
      return prisma.agentRun.findMany({
        where: { agentId: input.agentId },
        orderBy: { createdAt: "desc" },
        take: input.limit,
      });
    }),

  // Generate agent instructions from description
  generateInstructions: protectedProcedure
    .input(z.object({ description: z.string() }))
    .mutation(async ({ input }) => {
      const result = await aiService.generateText(
        `Generate detailed instructions for an AI agent with
this description: "${input.description}"

Write clear, specific instructions telling the agent:
1. What its role and purpose is
2. What data it should read and how to process it
3. What actions it should take
4. How it should format its responses
```

```
5. Any rules or boundaries

Write in second person ("You are...", "You should...").`,
        {
          systemPrompt: "You are an expert at writing AI agen
t instructions. Be specific and clear.",
          maxTokens: 1000,
          temperature: 0.5,
        }
      );
      return { instructions: result.text };
    }),

  // Create from template
  createFromTemplate: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      templateId: z.enum(["sprint_status", "qa_knowledge", "m
eeting_prep", "onboarding", "task_triage", "weekly_digest"]),
    }))
    .mutation(async ({ input, ctx }) => {
      const TEMPLATES: Record<string, any> = {
        sprint_status: {
          name: "Sprint Status Bot",
          description: "Generates daily sprint status reports
and posts them in a channel",
          systemPrompt: "You are a Sprint Status Bot. Read al
l task changes from the last 24 hours. Group by team member:
completed, in-progress, blocked. Generate a clear, formatted
status report. Include sprint progress percentage and any ris
ks.",
          capabilities: ["read_tasks", "send_messages", "sear
ch_workspace"],
          triggerType: "scheduled",
          triggerConfig: { cron: "0 9 * * 1-5" },
        },
```

```
        qa_knowledge: {
          name: "Q&A Knowledge Bot",
          description: "Answers questions using workspace kno
wledge base",
          systemPrompt: "You are a Knowledge Bot. When someon
e asks a question, search the workspace for relevant informat
ion and provide a clear answer with citations to the source d
ocuments.",
          capabilities: ["search_workspace", "read_pages", "r
ead_messages"],
          triggerType: "message_in_channel",
          triggerConfig: { keywords: ["?", "how", "what", "wh
ere", "when", "who", "why"] },
        },
        meeting_prep: {
          name: "Meeting Prep Agent",
          description: "Gathers context before meetings",
          systemPrompt: "You are a Meeting Prep Agent. Before
each meeting, gather: recent task updates related to meeting
topic, open action items from previous meetings, relevant cha
nnel discussions. Create a concise prep brief.",
          capabilities: ["read_tasks", "read_messages", "read
_meetings", "read_calendar", "send_messages"],
          triggerType: "scheduled",
          triggerConfig: { minutesBefore: 30 },
        },
        onboarding: {
          name: "Onboarding Agent",
          description: "Welcomes and guides new team member
s",
          systemPrompt: "You are an Onboarding Agent. When a
new member joins, send a welcome DM with: workspace overview,
key channels to join, important documents to read, team intro
ductions, and first tasks.",
          capabilities: ["send_messages", "read_pages", "sear
ch_workspace", "send_notifications"],
```

```
          triggerType: "event",
          triggerConfig: { eventType: "member_joined" },
        },
        task_triage: {
          name: "Task Triage Agent",
          description: "Auto-prioritizes and categorizes new
tasks",
          systemPrompt: "You are a Task Triage Agent. When a
new task is created, read its title and description. Assign:
priority (Low/Medium/High/Critical), category (Bug/Feature/En
hancement/Documentation), and suggest the best assignee based
on who usually handles similar tasks.",
          capabilities: ["read_tasks", "write_tasks", "search
_workspace"],
          triggerType: "event",
          triggerConfig: { eventType: "task_created" },
        },
        weekly_digest: {
          name: "Weekly Digest Agent",
          description: "Friday afternoon summary of the wee
k",
          systemPrompt: "You are a Weekly Digest Agent. Every
Friday at 4 PM, compile: tasks completed this week, key decis
ions made, meetings held, upcoming deadlines next week, and a
ny open blockers. Post as a formatted summary in the team cha
nnel.",
          capabilities: ["read_tasks", "read_messages", "read
_meetings", "send_messages", "search_workspace"],
          triggerType: "scheduled",
          triggerConfig: { cron: "0 16 * * 5" },
        },
      };

      const template = TEMPLATES[input.templateId];
      if (!template) throw new Error("Template not found");
```

```
      return prisma.aiAgent.create({
        data: {
          ...template,
          workspaceId: input.workspaceId,
          createdBy: ctx.user.id,
          type: "template",
          model: "gpt-4o",
          temperature: 0.5,
          visibility: "workspace",
          visibleToUserIds: [],
          dataAccess: { allPages: true, allDatabases: true, a
llChannels: true, allMeetings: true, specificPageIds: [], spe
cificDatabaseIds: [], specificChannelIds: [] },
          isActive: true,
          maxRunsPerDay: 100,
          maxActionsPerRun: 25,
          timeoutMinutes: 5,
        },
      });
    }),
});

// ============================================================
==
// AGENT TOOL DEFINITIONS & EXECUTION
// ============================================================
==

function buildAgentTools(capabilities: string[]): any[] {
  const allTools: Record<string, any> = {
    read_pages: { type: "function", function: { name: "read_p
age", description: "Read the content of a page", parameters:
{ type: "object", properties: { pageId: { type: "string" } },
required: ["pageId"] } } },
    search_workspace: { type: "function", function: { name:
"search_workspace", description: "Search across all workspace
```

```
data", parameters: { type: "object", properties: { query: { t
ype: "string" } }, required: ["query"] } } },
    read_tasks: { type: "function", function: { name: "read_t
asks", description: "Read tasks from a database", parameters:
{ type: "object", properties: { filter: { type: "string", des
cription: "Optional filter like 'status:in_progress' or 'assi
gnee:alice'" } }, required: [] } } },
    write_tasks: { type: "function", function: { name: "creat
e_task", description: "Create a new task", parameters: { typ
e: "object", properties: { title: { type: "string" }, descrip
tion: { type: "string" }, priority: { type: "string" }, dueDa
te: { type: "string" } }, required: ["title"] } } },
    send_messages: { type: "function", function: { name: "sen
d_message", description: "Send a message to a channel", param
eters: { type: "object", properties: { channelId: { type: "st
ring" }, content: { type: "string" } }, required: ["channelI
d", "content"] } } },
    read_messages: { type: "function", function: { name: "rea
d_messages", description: "Read recent messages from a channe
l", parameters: { type: "object", properties: { channelId: {
type: "string" }, limit: { type: "number" } }, required: ["ch
annelId"] } } },
  };

  return capabilities.filter(c => allTools[c]).map(c => allTo
ols[c]);
}

async function executeAgentTool(toolName: string, args: any,
agent: any, userId: string): Promise<any> {
  switch (toolName) {
    case "search_workspace":
      const searchResults = await prisma.$queryRaw`
        SELECT "sourceType", title, LEFT(content, 300) as con
tent
        FROM "KnowledgeIndex"
```

```
        WHERE "workspaceId" = ${agent.workspaceId}::uuid
        AND to_tsvector('english', content) @@ plainto_tsquer
y('english', ${args.query})
        LIMIT 5
      `;
      return searchResults;

    case "read_tasks":
      return prisma.task.findMany({
        where: { workspaceId: agent.workspaceId, deletedAt: n
ull },
        take: 20,
        orderBy: { updatedAt: "desc" },
        select: { id: true, title: true, status: true, priori
ty: true, assigneeId: true, dueDate: true },
      });

    case "send_message":
      // Verify agent has access to this channel
      return { sent: true, channelId: args.channelId, conten
t: args.content };

    case "read_page":
      return prisma.page.findUnique({
        where: { id: args.pageId },
        select: { id: true, title: true, content: true },
      });

    default:
      return { error: `Unknown tool: ${toolName}` };
  }
}
```

Agent Chat UI: `src/components/agents/agent-chat.tsx`

Build a standard chat interface:

- Left panel: list of agents with avatars, names, last message preview
- Right panel: conversation view with user messages (right-aligned) and agent messages (left-aligned)
- Bottom: message input with send button
- Each agent message shows: text response + collapsible "Steps" showing what tools the agent used
- "New Agent" button opens the agent builder form

Agent Builder UI: `src/components/agents/agent-builder.tsx`

Build a multi-section form:

- Name, Icon, Description fields
- Instructions textarea with "Generate from description" button
- Model selector dropdown
- Capabilities: checkboxes for each capability
- Data Access: radio buttons (All / Specific) with multi-select for specific items
- Trigger: radio buttons (Manual / Scheduled / Event / Message / Webhook) with config per type
- Visibility: radio buttons (Workspace / Specific members / Private)
- Safety: max runs/day, max actions/run, timeout inputs
- "Create Agent" button

FEATURE 19: AI WORKFLOW RECORDER

tRPC Router: `src/server/routers/workflows.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
```

```
export const workflowsRouter = router({
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      return prisma.workflow.findMany({
        where: {
          workspaceId: input.workspaceId,
          deletedAt: null,
          OR: [{ isPublic: true }, { createdBy: ctx.user.id
}],
        },
        orderBy: { createdAt: "desc" },
      });
    }),

  // Save recorded workflow steps
  saveRecording: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      name: z.string().min(1),
      description: z.string().optional(),
      recordedSteps: z.array(z.object({
        stepNumber: z.number(),
        actionType: z.string(),
        rawData: z.any(),
      })),
    }))
    .mutation(async ({ input, ctx }) => {
      // AI cleans up the raw steps and detects variables
      const aiResult = await aiService.generateJSON<{
        cleanedSteps: Array<{ stepNumber: number; actionType:
string; cleanedData: any; variables: Array<{ name: string; ty
pe: string; defaultValue?: string }> }>;
        variables: Array<{ name: string; type: string; descri
ption: string }>;
      }>(
```

```
        `Clean up these recorded workflow steps and detect re
usable variables.

RECORDED STEPS:
${JSON.stringify(input.recordedSteps, null, 2)}

For each step:
1. Clean the data (remove unnecessary fields)
2. Detect variables (names, dates, project names, user refere
nces that would change each time)
3. Replace hardcoded values with variable references like {{v
ariable_name}}

Return: { cleanedSteps: [...], variables: [...] }`,
        { systemPrompt: "You are an automation expert. Clean
workflow recordings and extract variables.", temperature: 0.2
},
      );

      return prisma.workflow.create({
        data: {
          workspaceId: input.workspaceId,
          name: input.name,
          description: input.description,
          recordedSteps: aiResult.cleanedSteps,
          variables: aiResult.variables,
          createdBy: ctx.user.id,
        },
      });
    }),

  // Execute a workflow with variable values
  execute: protectedProcedure
    .input(z.object({
      workflowId: z.string().uuid(),
      variableValues: z.record(z.string()),
```

```
    }))
    .mutation(async ({ input, ctx }) => {
      const workflow = await prisma.workflow.findUnique({ whe
re: { id: input.workflowId } });
      if (!workflow) throw new Error("Workflow not found");

      const run = await prisma.workflowRun.create({
        data: {
          workflowId: workflow.id,
          userId: ctx.user.id,
          variableValues: input.variableValues,
          steps: [],
          status: "running",
        },
      });

      // Execute each step
      const steps = workflow.recordedSteps as any[];
      const executedSteps: any[] = [];

      for (const step of steps) {
        try {
          // Replace variables in step data
          let stepData = JSON.stringify(step.cleanedData || s
tep.rawData);
          for (const [key, value] of Object.entries(input.var
iableValues)) {
            stepData = stepData.replace(new RegExp(`{{${ke
y}}}`, "g"), value);
          }
          const parsedData = JSON.parse(stepData);

          // Execute the action (adapt to your actual action
types)
          const result = await executeWorkflowStep(step.actio
nType, parsedData, workflow.workspaceId, ctx.user.id);
```

```
          executedSteps.push({ stepNumber: step.stepNumber, s
tatus: "success", output: result });
        } catch (error: any) {
          executedSteps.push({ stepNumber: step.stepNumber, s
tatus: "failed", error: error.message });
        }
      }

      await prisma.workflowRun.update({
        where: { id: run.id },
        data: { steps: executedSteps, status: "completed", co
mpletedAt: new Date() },
      });

      await prisma.workflow.update({
        where: { id: workflow.id },
        data: { runCount: { increment: 1 } },
      });

      return { runId: run.id, steps: executedSteps };
    }),

  delete: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.workflow.update({ where: { id: input.id
}, data: { deletedAt: new Date() } });
    }),
});

async function executeWorkflowStep(actionType: string, data:
any, workspaceId: string, userId: string): Promise<any> {
  switch (actionType) {
    case "create_task":
      return prisma.task.create({ data: { ...data, workspaceI
d, createdBy: userId } });
```

```
    case "create_page":
      return prisma.page.create({ data: { ...data, workspaceI
d, createdBy: userId } });
    case "send_message":
      // Use your messaging service
      return { sent: true };
    case "update_task":
      return prisma.task.update({ where: { id: data.taskId },
data: data.updates });
    default:
      throw new Error(`Unknown action type: ${actionType}`);
  }
}
```

EDGE CASES — SPRINT 4

```
Agents:
- Agent loops (calls same tool repeatedly) → Max 10 tool-call
rounds, then force complete
- Agent tries to access data outside its config → Permission
check rejects, log attempt
- Agent timeout → Kill after 5 min, status = 'timeout', notif
y user
- Scheduled agent fires but workspace has no relevant data →
Agent responds "No new data to report"
- Multiple users chat with same shared agent → Separate conve
rsations per user

Workflows:
- Recorded workflow has deleted task/page → Skip step, show w
arning "Step 3 skipped: original task was deleted"
- Variable value empty → Show validation error before executi
on
```

```
- Workflow has 0 steps → Prevent saving, show "Record at leas
t one action"
```

VERIFICATION — SPRINT 4

```
□ Create agent from "Sprint Status Bot" template → agent appe
ars in list
□ Chat with agent: "What tasks are due this week?" → agent se
arches and responds
□ Agent shows tool usage in collapsible steps panel
□ Create custom agent with scheduled trigger → save successfu
lly
□ Agent builder form validates all required fields
□ Record a workflow (create task + send message) → save with
detected variables
□ Execute workflow with variables filled in → steps execute,
results shown
□ View workflow run history → see past executions with succes
s/fail per step
```

SPRINT 5 — AI PROJECT MANAGER + TASK INTELLIGENCE + DATABASE AI

Features 5, 6, and 8

PREREQUISITE: Sprints 1-4 complete

FEATURE 5: AI PROJECT MANAGER

tRPC Router: `src/server/routers/project-manager.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import { logAiUsage } from "@/server/services/ai-usage-logge
r";

export const projectManagerRouter = router({
  // Generate standup report
  generateStandup: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), databas
eId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const yesterday = new Date();
      yesterday.setDate(yesterday.getDate() - 1);

      // Get task changes in last 24 hours
      const recentTasks = await prisma.task.findMany({
        where: { workspaceId: input.workspaceId, databaseId:
input.databaseId, updatedAt: { gte: yesterday } },
        include: { assignee: { select: { id: true, name: true
} } },
      });

      const completedTasks = recentTasks.filter(t => t.status
=== "done" || t.completedAt);
      const inProgressTasks = recentTasks.filter(t => t.statu
s === "in_progress");
      const blockedTasks = recentTasks.filter(t => t.status =
== "blocked");
      const overdueTasks = await prisma.task.findMany({
        where: { databaseId: input.databaseId, dueDate: { lt:
new Date() }, completedAt: null, deletedAt: null },
        include: { assignee: { select: { name: true } } },
      });
```

```
      const totalTasks = await prisma.task.count({ where: { d
atabaseId: input.databaseId, deletedAt: null } });
      const doneTasks = await prisma.task.count({ where: { da
tabaseId: input.databaseId, completedAt: { not: null }, delet
edAt: null } });
      const progressPercent = totalTasks > 0 ? Math.round((do
neTasks / totalTasks) * 100) : 0;

      const result = await aiService.generateText(
        `Generate a daily standup report from this data:

COMPLETED (last 24h): ${JSON.stringify(completedTasks.map(t =
> ({ title: t.title, assignee: t.assignee?.name })))}
IN PROGRESS: ${JSON.stringify(inProgressTasks.map(t => ({ tit
le: t.title, assignee: t.assignee?.name })))}
BLOCKED: ${JSON.stringify(blockedTasks.map(t => ({ title: t.t
itle, assignee: t.assignee?.name })))}
OVERDUE: ${JSON.stringify(overdueTasks.map(t => ({ title: t.t
itle, assignee: t.assignee?.name, daysOverdue: Math.ceil((Dat
e.now() - (t.dueDate?.getTime() || 0)) / 86400000) }))}
PROGRESS: ${progressPercent}% complete (${doneTasks}/${totalT
asks} tasks)

Format as a clear standup grouped by team member. Include spr
int progress and any risks.`,
        { systemPrompt: "Generate a concise, well-formatted s
tandup report.", maxTokens: 1500, temperature: 0.3 },
      );

      const report = await prisma.projectStatusReport.create
({
        data: {
          workspaceId: input.workspaceId,
          databaseId: input.databaseId,
          generatedBy: "ai_on_demand",
```

```
          reportType: "daily_standup",
          periodStart: yesterday,
          periodEnd: new Date(),
          content: {
            summary: result.text,
            completedTasks: completedTasks.map(t => ({ taskI
d: t.id, title: t.title, completedBy: t.assignee?.name })),
            inProgressTasks: inProgressTasks.map(t => ({ task
Id: t.id, title: t.title, assignee: t.assignee?.name })),
            blockedTasks: blockedTasks.map(t => ({ taskId: t.
id, title: t.title, assignee: t.assignee?.name })),
            overdueTasks: overdueTasks.map(t => ({ taskId: t.
id, title: t.title, assignee: t.assignee?.name })),
            progressPercent,
          },
        },
      });

      return { report, summary: result.text };
    }),

  // Detect project risks
  detectRisks: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), databas
eId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const tasks = await prisma.task.findMany({
        where: { databaseId: input.databaseId, completedAt: n
ull, deletedAt: null },
        include: { assignee: { select: { id: true, name: true
} } },
      });

      const risks: any[] = [];

      // 1. Deadline at risk
```

```
      for (const task of tasks) {
        if (task.dueDate) {
          const daysLeft = Math.ceil((task.dueDate.getTime()
- Date.now()) / 86400000);
          if (daysLeft <= 1 && task.status !== "in_progress")
{
            risks.push({
              riskType: "deadline_at_risk",
              severity: daysLeft <= 0 ? "critical" : "high",
              title: `"${task.title}" deadline ${daysLeft <=
0 ? "overdue" : "tomorrow"}`,
              description: `Task "${task.title}" is due ${day
sLeft <= 0 ? `${Math.abs(daysLeft)} days overdue` : "tomorro
w"} but status is "${task.status}".`,
              affectedTaskIds: [task.id],
              affectedUserIds: task.assigneeId ? [task.assign
eeId] : [],
              suggestedAction: daysLeft <= 0 ? "Reassign or e
scalate immediately." : "Move to in-progress and prioritiz
e.",
            });
          }
        }
      }

      // 2. Team member overloaded
      const assigneeCounts: Record<string, { count: number; n
ame: string; userId: string }> = {};
      for (const task of tasks) {
        if (task.assigneeId && task.assignee) {
          if (!assigneeCounts[task.assigneeId]) assigneeCount
s[task.assigneeId] = { count: 0, name: task.assignee.name, us
erId: task.assigneeId };
          assigneeCounts[task.assigneeId].count++;
        }
      }
```

```
      for (const [userId, data] of Object.entries(assigneeCou
nts)) {
        if (data.count > 8) {
          risks.push({
            riskType: "team_member_overloaded",
            severity: data.count > 12 ? "high" : "medium",
            title: `${data.name} has ${data.count} active tas
ks`,
            description: `${data.name} is assigned ${data.cou
nt} active tasks, which exceeds the recommended limit of 8.`,
            affectedTaskIds: tasks.filter(t => t.assigneeId =
== userId).map(t => t.id),
            affectedUserIds: [userId],
            suggestedAction: `Consider redistributing ${data.
count - 8} tasks to team members with lighter workloads.`,
          });
        }
      }

      // 3. Unassigned urgent
      const unassignedUrgent = tasks.filter(t => !t.assigneeI
d && (t.priority === "high" || t.priority === "urgent"));
      if (unassignedUrgent.length > 0) {
        risks.push({
          riskType: "unassigned_urgent",
          severity: "medium",
          title: `${unassignedUrgent.length} urgent tasks hav
e no assignee`,
          description: `${unassignedUrgent.map(t => `"${t.tit
le}"`).join(", ")} need owners.`,
          affectedTaskIds: unassignedUrgent.map(t => t.id),
          affectedUserIds: [],
          suggestedAction: "Assign these tasks immediately ba
sed on team expertise and availability.",
        });
      }
```

```
      // Save risks
      for (const risk of risks) {
        await prisma.aiRiskAlert.upsert({
          where: { id: risk.id || "00000000-0000-0000-0000-00
0000000000" },
          create: { ...risk, workspaceId: input.workspaceId,
databaseId: input.databaseId },
          update: risk,
        });
      }

      return { risks, count: risks.length };
    }),

  // Get active risks
  getRisks: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), databas
eId: z.string().uuid().optional() }))
    .query(async ({ input }) => {
      return prisma.aiRiskAlert.findMany({
        where: {
          workspaceId: input.workspaceId,
          ...(input.databaseId ? { databaseId: input.database
Id } : {}),
          status: "active",
        },
        orderBy: [{ severity: "asc" }, { createdAt: "desc"
}],
      });
    }),

  // Generate status report
  generateStatusReport: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), databas
eId: z.string().uuid() }))
```

```
    .mutation(async ({ input }) => {
      const tasks = await prisma.task.findMany({
        where: { databaseId: input.databaseId, deletedAt: nul
l },
        include: { assignee: { select: { name: true } } },
      });

      const total = tasks.length;
      const done = tasks.filter(t => t.completedAt).length;
      const inProgress = tasks.filter(t => t.status === "in_p
rogress").length;
      const blocked = tasks.filter(t => t.status === "blocke
d").length;
      const overdue = tasks.filter(t => t.dueDate && t.dueDat
e < new Date() && !t.completedAt).length;

      const result = await aiService.generateText(
        `Generate an executive project status report:
Total tasks: ${total}, Done: ${done} (${Math.round(done/total
*100)}%), In Progress: ${inProgress}, Blocked: ${blocked}, Ov
erdue: ${overdue}

Task list: ${JSON.stringify(tasks.map(t => ({ title: t.title,
status: t.status, assignee: t.assignee?.name, dueDate: t.dueD
ate, priority: t.priority })))}

Write a concise executive summary (3-4 sentences), then list
key highlights and concerns.`,
        { systemPrompt: "Write a professional project status
report for stakeholders.", maxTokens: 1000, temperature: 0.3
},
      );

      return { report: result.text };
    }),
```

```
  // Create automation from natural language
  createAutomation: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      description: z.string(),
    }))
    .mutation(async ({ input, ctx }) => {
      const result = await aiService.generateJSON<{
        name: string; trigger: any; conditions: any[]; action
s: any[];
      }>(
        `Parse this automation description into a structured
automation config:
"${input.description}"

Return JSON with: name, trigger ({type, config}), conditions
([{field, operator, value}]), actions ([{type, config}])
Trigger types: task_created, task_status_changed, task_overdu
e, scheduled
Action types: update_task, send_message, create_task, assign,
notify`,
        { systemPrompt: "Parse natural language into automati
on configs.", temperature: 0.2 },
      );

      return prisma.aiWorkflowAutomation.create({
        data: {
          workspaceId: input.workspaceId,
          name: result.name,
          createdFromPrompt: input.description,
          trigger: result.trigger,
          conditions: result.conditions,
          actions: result.actions,
          createdBy: ctx.user.id,
        },
      });
```

```
    }),
});
```

FEATURE 6: AI TASK INTELLIGENCE

tRPC Router: `src/server/routers/task-intelligence.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const taskIntelligenceRouter = router({
  // AI Task Breakdown (Goblin.tools-style)
  breakdown: protectedProcedure
    .input(z.object({
      taskTitle: z.string(),
      taskDescription: z.string().optional(),
      complexity: z.enum(["simple", "medium", "detailed", "ve
ry_detailed"]).default("medium"),
    }))
    .mutation(async ({ input }) => {
      const depthMap = { simple: "3-5", medium: "5-8", detail
ed: "8-12", very_detailed: "12-20" };
      const result = await aiService.generateJSON<{ subtasks:
Array<{ title: string; subtasks?: Array<{ title: string }> }>
}>(
        `Break down this task into ${depthMap[input.complexit
y]} actionable subtasks:
Task: "${input.taskTitle}"
${input.taskDescription ? `Description: ${input.taskDescripti
on}` : ""}

Return JSON: { subtasks: [{ title: string, subtasks?: [{ titl
```

```
e: string }] }] }
Each subtask should be a clear, actionable item one person ca
n complete.`,
        { systemPrompt: "Break tasks into clear, actionable s
ubtasks.", temperature: 0.4 },
      );
      return result;
    }),

  // AI Prioritize all tasks in a database
  prioritize: protectedProcedure
    .input(z.object({ databaseId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const tasks = await prisma.task.findMany({
        where: { databaseId: input.databaseId, completedAt: n
ull, deletedAt: null },
        select: { id: true, title: true, description: true, d
ueDate: true, priority: true, status: true },
      });

      const result = await aiService.generateJSON<{ ranked: A
rray<{ taskId: string; suggestedPriority: string; reason: str
ing }> }>(
        `Rank these tasks by priority (critical > high > medi
um > low):
${JSON.stringify(tasks)}

Consider: due date urgency, task complexity, blocking potenti
al, business impact.
Return: { ranked: [{ taskId, suggestedPriority: 'critical'|'h
igh'|'medium'|'low', reason: string }] }`,
        { systemPrompt: "Prioritize tasks based on urgency an
d impact.", temperature: 0.2 },
      );
      return result;
    }),
```

```
  // Natural language task creation
  parseNaturalLanguage: protectedProcedure
    .input(z.object({ text: z.string(), workspaceId: z.string
().uuid() }))
    .mutation(async ({ input }) => {
      const result = await aiService.generateJSON<{
        title: string; description?: string; dueDate?: strin
g; priority?: string;
        project?: string; assignee?: string; estimatedMinute
s?: number;
      }>(
        `Parse this natural language into a structured task:
"${input.text}"

Extract: title, description, dueDate (ISO format), priority
(low/medium/high/urgent), project name (if mentioned), assign
ee name (if mentioned), estimatedMinutes.
Return JSON. Only include fields that are explicitly mentione
d or clearly implied.`,
        { systemPrompt: "Parse natural language into structur
ed task data.", temperature: 0.2 },
      );
      return result;
    }),

  // AI Duration Estimation
  estimateDuration: protectedProcedure
    .input(z.object({ taskId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const task = await prisma.task.findUnique({ where: { i
d: input.taskId } });
      if (!task) throw new Error("Task not found");

      const result = await aiService.generateJSON<{ estimated
Minutes: number; confidence: number; reasoning: string }>(
```

```
        `Estimate how long this task will take:
Title: "${task.title}"
Description: "${task.description || "No description"}"

Return: { estimatedMinutes: number (round to nearest 15), con
fidence: 0-1, reasoning: string }`,
        { systemPrompt: "Estimate task durations based on com
plexity.", temperature: 0.3 },
      );

      await prisma.taskDurationEstimate.upsert({
        where: { taskId: task.id },
        create: { taskId: task.id, workspaceId: task.workspac
eId, estimatedMinutes: result.estimatedMinutes, confidence: r
esult.confidence },
        update: { estimatedMinutes: result.estimatedMinutes,
confidence: result.confidence },
      });

      return result;
    }),

  // AI Assignee Suggestion
  suggestAssignee: protectedProcedure
    .input(z.object({ taskId: z.string().uuid(), workspaceId:
z.string().uuid() }))
    .mutation(async ({ input }) => {
      const task = await prisma.task.findUnique({ where: { i
d: input.taskId } });
      if (!task) throw new Error("Task not found");

      const members = await prisma.workspaceMember.findMany({
        where: { workspaceId: input.workspaceId },
        include: { user: { select: { id: true, name: true } }
},
      });
```

```
      // Get task counts per member
      const memberWorkloads = await Promise.all(
        members.map(async (m) => {
          const activeCount = await prisma.task.count({
            where: { assigneeId: m.userId, completedAt: null,
deletedAt: null },
          });
          return { userId: m.userId, name: m.user.name, activ
eTasks: activeCount };
        })
      );

      const result = await aiService.generateJSON<{ suggestio
ns: Array<{ userId: string; name: string; score: number; reas
on: string }> }>(
        `Suggest the best assignee for this task:
Task: "${task.title}" - ${task.description || "No descriptio
n"}

Team members and their current workload:
${JSON.stringify(memberWorkloads)}

Return top 3 suggestions with score (0-100) and reasoning.`,
        { systemPrompt: "Suggest task assignees based on skil
ls and workload.", temperature: 0.3 },
      );
      return result;
    }),
});
```

FEATURE 8: AI DATABASE INTELLIGENCE

tRPC Router: `src/server/routers/database-ai.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const databaseAiRouter = router({
  // AI Auto-fill for a column
  autoFill: protectedProcedure
    .input(z.object({
      databaseId: z.string().uuid(),
      taskId: z.string().uuid(),
      columnId: z.string().uuid(),
    }))
    .mutation(async ({ input }) => {
      const column = await prisma.aiAutoFillColumn.findUnique
({ where: { id: input.columnId } });
      if (!column) throw new Error("Auto-fill column not foun
d");

      const task = await prisma.task.findUnique({ where: { i
d: input.taskId } });
      if (!task) throw new Error("Task not found");

      // Build input from specified fields
      const inputData: Record<string, string> = {};
      for (const field of column.inputFields) {
        inputData[field] = (task as any)[field] || "";
      }

      const result = await aiService.generateText(
        `${column.aiPrompt}\n\nInput data:\n${JSON.stringify
(inputData)}`,
        { systemPrompt: "Generate the requested value based o
n the input data. Return ONLY the value, nothing else.", maxT
okens: 200, temperature: 0.2 },
```

```
      );

      return { value: result.text.trim() };
    }),

  // Natural language database query
  queryDatabase: protectedProcedure
    .input(z.object({ databaseId: z.string().uuid(), query:
z.string() }))
    .mutation(async ({ input }) => {
      const tasks = await prisma.task.findMany({
        where: { databaseId: input.databaseId, deletedAt: nul
l },
        include: { assignee: { select: { name: true } } },
        take: 100,
      });

      const result = await aiService.generateText(
        `Answer this question about the database:
"${input.query}"

DATABASE CONTENTS (${tasks.length} rows):
${JSON.stringify(tasks.map(t => ({ title: t.title, status: t.
status, priority: t.priority, assignee: t.assignee?.name, due
Date: t.dueDate, createdAt: t.createdAt })))}

Provide a direct answer. If it's a count, give the number. If
it's a list, list the items.`,
        { systemPrompt: "Answer database queries accurately a
nd concisely.", maxTokens: 500, temperature: 0.2 },
      );
      return { answer: result.text };
    }),

  // AI Database Creation from description
  createFromDescription: protectedProcedure
```

```
    .input(z.object({ workspaceId: z.string().uuid(), descrip
tion: z.string() }))
    .mutation(async ({ input, ctx }) => {
      const result = await aiService.generateJSON<{
        name: string;
        properties: Array<{ name: string; type: string; optio
ns?: string[] }>;
        views: Array<{ name: string; type: string; groupBy?:
string }>;
        sampleRows: Array<Record<string, any>>;
      }>(
        `Create a database structure for: "${input.descriptio
n}"

Return JSON with:
- name: database name
- properties: [{name, type: 'text'|'select'|'date'|'numbe
r'|'person'|'checkbox'|'url'|'email', options?: [] for select
type}]
- views: [{name, type: 'table'|'board'|'calendar', groupBy?:
property name for board view}]
- sampleRows: 3 example rows with data

Make it practical and complete.`,
        { systemPrompt: "Design practical database structure
s.", temperature: 0.4 },
      );
      return result;
    }),

  // AI Formula Generation
  generateFormula: protectedProcedure
    .input(z.object({ description: z.string(), properties: z.
array(z.object({ name: z.string(), type: z.string() })) }))
    .mutation(async ({ input }) => {
      const result = await aiService.generateText(
```

```
        `Generate a formula for: "${input.description}"
Available properties: ${JSON.stringify(input.properties)}
Return ONLY the formula expression, nothing else.`,
        { systemPrompt: "Generate database formulas.", maxTok
ens: 200, temperature: 0.1 },
      );
      return { formula: result.text.trim() };
    }),

  // CRUD for auto-fill columns
  createAutoFillColumn: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(), databaseId: z.string().
uuid(),
      columnName: z.string(), aiPrompt: z.string(), inputFiel
ds: z.array(z.string()),
      outputType: z.enum(["text", "select", "number"]).defaul
t("text"),
      selectOptions: z.array(z.string()).optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.aiAutoFillColumn.create({
        data: { ...input, createdBy: ctx.user.id },
      });
    }),
});
```

VERIFICATION — SPRINT 5

```
□ Click "Generate Standup" on project board → formatted stand
up report appears
□ Click "Detect Risks" → risk cards shown with severity level
s and suggestions
□ Click "Generate Status Report" → executive summary generate
d from real task data
```

```
☐ Type natural language automation → AI parses → preview show
n → confirm creates automation
☐ Click "Break Down" on a task → subtask tree generated with
adjustable complexity
☐ Click "AI Prioritize" on database → all tasks get priority
suggestions with reasoning
☐ Type "Review Alice's PR tomorrow high priority" → AI parses
into structured task fields
☐ Click "Estimate Duration" → shows "~45 min" with confidence
score
☐ Click "Suggest Assignee" → top 3 members with scores and re
asons
☐ Add AI Auto-Fill column → new rows auto-populate with AI-ge
nerated values
☐ Type natural language query on database → get accurate answ
er
☐ Describe a database → AI creates with properties, views, sa
mple data
```

SPRINT 6 — MEETING INTELLIGENCE + DOCUMENT GENERATOR

Features 7 and 10

PREREQUISITE: Sprints 1-5 complete

FEATURE 7: AI MEETING INTELLIGENCE

tRPC Router: `src/server/routers/meetings.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import { meetingQueue } from "@/jobs/queue-config";
import { indexTask } from "@/server/services/knowledge-indexe
r";

export const meetingsRouter = router({
  // Process meeting (transcription + AI summary) — triggered
after recording ends
  process: protectedProcedure
    .input(z.object({ meetingId: z.string().uuid(), audioUrl:
z.string() }))
    .mutation(async ({ input }) => {
      // Create intelligence record
      const intel = await prisma.meetingIntelligence.create({
        data: {
          meetingId: input.meetingId,
          workspaceId: (await prisma.meeting.findUnique({ whe
re: { id: input.meetingId } }))!.workspaceId,
          audioUrl: input.audioUrl,
          processingStatus: "transcribing",
        },
      });

      // Enqueue for background processing
      await meetingQueue.add("process-meeting", {
        intelligenceId: intel.id,
        meetingId: input.meetingId,
        audioUrl: input.audioUrl,
      });

      return { intelligenceId: intel.id, status: "processing"
};
```

```
    }),

  // Get meeting intelligence (summary, action items, etc.)
  getIntelligence: protectedProcedure
    .input(z.object({ meetingId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.meetingIntelligence.findUnique({
        where: { meetingId: input.meetingId },
      });
    }),

  // Create tasks from action items
  createTasksFromActionItems: protectedProcedure
    .input(z.object({
      meetingId: z.string().uuid(),
      databaseId: z.string().uuid(),
      actionItemIndices: z.array(z.number()), // which action
items to create tasks for
    }))
    .mutation(async ({ input, ctx }) => {
      const intel = await prisma.meetingIntelligence.findUniq
ue({ where: { meetingId: input.meetingId } });
      if (!intel || !intel.actionItems) throw new Error("No a
ction items found");

      const actionItems = intel.actionItems as any[];
      const createdTasks: any[] = [];

      for (const index of input.actionItemIndices) {
        const item = actionItems[index];
        if (!item) continue;

        const task = await prisma.task.create({
          data: {
            title: item.title,
            description: item.description || `Action item fro
```

```
m meeting`,
            workspaceId: intel.workspaceId,
            databaseId: input.databaseId,
            createdBy: ctx.user.id,
            dueDate: item.suggestedDueDate ? new Date(item.su
ggestedDueDate) : undefined,
          },
        });

        // Update action item to show task was created
        actionItems[index].taskCreated = true;
        actionItems[index].taskId = task.id;
        createdTasks.push(task);

        // Index the new task for knowledge search
        await indexTask(task.id, intel.workspaceId);
      }

      // Update meeting intelligence with created task IDs
      await prisma.meetingIntelligence.update({
        where: { meetingId: input.meetingId },
        data: { actionItems: actionItems },
      });

      return { created: createdTasks.length, tasks: createdTa
sks };
    }),

  // Generate meeting prep brief
  generatePrepBrief: protectedProcedure
    .input(z.object({ meetingId: z.string().uuid(), workspace
Id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const meeting = await prisma.meeting.findUnique({ wher
e: { id: input.meetingId } });
      if (!meeting) throw new Error("Meeting not found");
```

```
      // Gather context: recent tasks, last meeting's action
items, relevant channel discussions
      const recentTasks = await prisma.task.findMany({
        where: { workspaceId: input.workspaceId, updatedAt: {
gte: new Date(Date.now() - 7 * 86400000) } },
        take: 15,
        orderBy: { updatedAt: "desc" },
      });

      const result = await aiService.generateText(
        `Generate a meeting prep brief for: "${meeting.titl
e}"
Meeting time: ${meeting.startTime}
Recent task updates: ${JSON.stringify(recentTasks.map(t => ({
title: t.title, status: t.status })))}

Include: agenda suggestions, relevant context, open questions
to discuss.`,
        { systemPrompt: "Generate concise meeting prep brief
s.", maxTokens: 800, temperature: 0.4 },
      );

      await prisma.meetingIntelligence.upsert({
        where: { meetingId: input.meetingId },
        create: { meetingId: input.meetingId, workspaceId: in
put.workspaceId, prepBrief: result.text },
        update: { prepBrief: result.text },
      });

      return { prepBrief: result.text };
    }),
});
```

Background Job: `src/jobs/process-meeting.ts`

```
// src/jobs/process-meeting.ts
import { Worker } from "bullmq";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import OpenAI from "openai";
import { redis } from "@/lib/redis";

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KE
Y });

const worker = new Worker("meeting", async (job) => {
  const { intelligenceId, meetingId, audioUrl } = job.data;

  try {
    // Step 1: Transcribe with Whisper
    await prisma.meetingIntelligence.update({
      where: { id: intelligenceId },
      data: { processingStatus: "transcribing" },
    });

    // Download audio and send to Whisper
    const audioResponse = await fetch(audioUrl);
    const audioBuffer = await audioResponse.arrayBuffer();
    const audioFile = new File([audioBuffer], "meeting.webm",
{ type: "audio/webm" });

    const transcription = await openai.audio.transcriptions.c
reate({
      model: "whisper-1",
      file: audioFile,
      response_format: "verbose_json",
      timestamp_granularities: ["segment"],
    });

    const transcript = transcription.text;
```

```
    const segments = (transcription as any).segments || [];

    // Step 2: Generate AI summary
    await prisma.meetingIntelligence.update({
      where: { id: intelligenceId },
      data: { processingStatus: "summarizing", transcriptRaw:
transcript, transcriptSegments: segments },
    });

    const summaryResult = await aiService.generateJSON<{
      summary: string;
      keyPoints: string[];
      decisions: Array<{ decision: string; context: string }
>;
      actionItems: Array<{ title: string; description?: strin
g; suggestedOwner?: string; suggestedDueDate?: string }>;
      openQuestions: string[];
    }>(
      `Analyze this meeting transcript and extract structured
information:

TRANSCRIPT:
${transcript.slice(0, 15000)}

Return JSON with: summary (3-5 sentences), keyPoints (array o
f strings), decisions (array of {decision, context}), actionI
tems (array of {title, description, suggestedOwner, suggested
DueDate}), openQuestions (array of strings).`,
      { systemPrompt: "Extract structured meeting information
from transcripts.", temperature: 0.2 },
    );

    // Step 3: Save everything
    await prisma.meetingIntelligence.update({
      where: { id: intelligenceId },
      data: {
```

```
        processingStatus: "completed",
        aiSummary: summaryResult.summary,
        keyPoints: summaryResult.keyPoints,
        decisions: summaryResult.decisions,
        actionItems: summaryResult.actionItems.map(a => ({
...a, taskCreated: false })),
        openQuestions: summaryResult.openQuestions,
        audioDurationSecs: Math.round(segments.length > 0 ? s
egments[segments.length - 1].end : 0),
      },
    });

    // Step 4: Index transcript for knowledge search
    const workspace = await prisma.meetingIntelligence.findUn
ique({
      where: { id: intelligenceId },
      select: { workspaceId: true },
    });
    if (workspace) {
      const { indexPage } = await import("@/server/services/k
nowledge-indexer");
      // Index as meeting_transcript type
    }

  } catch (error: any) {
    await prisma.meetingIntelligence.update({
      where: { id: intelligenceId },
      data: { processingStatus: "failed", processingError: er
ror.message },
    });
    throw error;
  }
}, { connection: redis });

export default worker;
```

FEATURE 10: AI DOCUMENT GENERATOR

tRPC Router: `src/server/routers/doc-generator.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const docGeneratorRouter = router({
  generate: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      templateType: z.enum([
        "weekly_status", "meeting_agenda", "project_brief",
"sprint_retro",
        "handoff_doc", "client_update", "onboarding_guide",
      ]),
      config: z.object({
        databaseId: z.string().uuid().optional(),
        channelId: z.string().uuid().optional(),
        dateRangeStart: z.string().optional(),
        dateRangeEnd: z.string().optional(),
        projectName: z.string().optional(),
      }),
    }))
    .mutation(async ({ input, ctx }) => {
      const { workspaceId, templateType, config } = input;

      // Gather real data based on template type
      let contextData = "";

      if (templateType === "weekly_status" || templateType ==
= "sprint_retro") {
```

```
        const startDate = config.dateRangeStart ? new Date(co
nfig.dateRangeStart) : new Date(Date.now() - 7 * 86400000);
        const endDate = config.dateRangeEnd ? new Date(confi
g.dateRangeEnd) : new Date();

        const tasks = config.databaseId
          ? await prisma.task.findMany({ where: { databaseId:
config.databaseId, updatedAt: { gte: startDate, lte: endDate
} }, include: { assignee: { select: { name: true } } }, take:
50 })
          : [];

        const completed = tasks.filter(t => t.completedAt &&
t.completedAt >= startDate);
        const inProgress = tasks.filter(t => t.status === "in
_progress");
        const blocked = tasks.filter(t => t.status === "block
ed");

        contextData = `Period: ${startDate.toLocaleDateString
()} to ${endDate.toLocaleDateString()}
Tasks completed: ${completed.length} (${completed.map(t => t.
title).join(", ")})
In progress: ${inProgress.length} (${inProgress.map(t => t.ti
tle).join(", ")})
Blocked: ${blocked.length} (${blocked.map(t => t.title).join
(", ")})`;
      }

      if (templateType === "meeting_agenda") {
        const recentTasks = config.databaseId
          ? await prisma.task.findMany({ where: { databaseId:
config.databaseId, status: { in: ["in_progress", "blocked"] }
}, take: 10 })
          : [];
        contextData = `Open tasks: ${JSON.stringify(recentTas
```

```
ks.map(t => ({ title: t.title, status: t.status })))}`;
      }

      const TEMPLATE_PROMPTS: Record<string, string> = {
        weekly_status: "Generate a weekly status report with:
Executive Summary, Completed This Week, In Progress, Blocked
Items, Risks, Plan for Next Week.",
        meeting_agenda: "Generate a meeting agenda with: Revi
ew Action Items, Status Updates, Discussion Topics, Decisions
Needed, Next Steps.",
        project_brief: `Generate a project brief for "${confi
g.projectName || "Project"}" with: Overview, Objectives, Requ
irements, Timeline, Team, Risks.`,
        sprint_retro: "Generate a sprint retrospective with:
Sprint Summary, What Went Well, What Could Improve, Action It
ems for Next Sprint.",
        handoff_doc: "Generate a handoff document with: Proje
ct Overview, Current State, Key Decisions Made, Open Items, C
ontact Points, Important Resources.",
        client_update: "Generate a client-facing project upda
te with: Summary, Progress, Milestones, Next Steps, Timelin
e.",
        onboarding_guide: "Generate an onboarding guide with:
Welcome, Getting Started, Key Resources, Team Overview, First
Week Tasks.",
      };

      const result = await aiService.generateText(
        `${TEMPLATE_PROMPTS[templateType]}

WORKSPACE DATA:
${contextData}

Write a complete, professional document using the real data p
rovided.`,
        { systemPrompt: "Generate professional documents from
```

```
workspace data.", maxTokens: 3000, temperature: 0.4 },
      );

      return { content: result.text, templateType };
    }),
});
```

VERIFICATION — SPRINT 6

```
□ Record a meeting → stop → processing starts
□ After processing: summary, key points, decisions, action it
ems displayed
□ Click "Create All Tasks" from action items → tasks created
in database
□ Click "Generate Prep Brief" before meeting → brief shown wi
th context
□ Generate weekly status report → pulls real task data, creat
es formatted report
□ Generate meeting agenda → includes open tasks and action it
ems
□ Generate project brief → structured document with all secti
ons
□ All generated documents are editable in the document editor
```

SPRINT 7 — MESSAGING INTELLIGENCE + EMAIL ASSISTANT

Features 11 and 13

PREREQUISITE: Sprints 1-6 complete

FEATURE 11: AI MESSAGING INTELLIGENCE

tRPC Router: `src/server/routers/messaging-ai.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const messagingAiRouter = router({
  // Summarize a thread
  summarizeThread: protectedProcedure
    .input(z.object({ threadParentId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const messages = await prisma.message.findMany({
        where: { OR: [{ id: input.threadParentId }, { threadP
arentId: input.threadParentId }] },
        include: { author: { select: { name: true } } },
        orderBy: { createdAt: "asc" },
      });

      const result = await aiService.generateText(
        `Summarize this thread in 3-5 sentences:\n\n${message
s.map(m => `${m.author.name}: ${m.content}`).join("\n")}`,
        { systemPrompt: "Summarize message threads concisel
y.", maxTokens: 300, temperature: 0.3 },
      );
      return { summary: result.text };
    }),

  // Channel catch-up
  catchUp: protectedProcedure
    .input(z.object({ channelId: z.string().uuid(), since: z.
string() })) // ISO datetime
    .mutation(async ({ input }) => {
```

```
      const messages = await prisma.message.findMany({
        where: { channelId: input.channelId, createdAt: { gt
e: new Date(input.since) } },
        include: { author: { select: { name: true } } },
        orderBy: { createdAt: "asc" },
        take: 100,
      });

      if (messages.length === 0) return { summary: "Nothing n
ew since you were last here." };

      const result = await aiService.generateText(
        `Summarize what happened in this channel (${messages.
length} messages):\n\n${messages.map(m => `${m.author.name}:
${m.content}`).join("\n")}`,
        { systemPrompt: "Provide a catch-up summary. Highligh
t: key decisions, important updates, questions that need answ
ers, action items.", maxTokens: 500, temperature: 0.3 },
      );
      return { summary: result.text, messageCount: messages.l
ength };
    }),

  // Smart reply suggestions
  suggestReplies: protectedProcedure
    .input(z.object({ messageId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const message = await prisma.message.findUnique({
        where: { id: input.messageId },
        include: { author: { select: { name: true } } },
      });
      if (!message) throw new Error("Message not found");

      const result = await aiService.generateJSON<{ replies:
string[] }>(
        `Suggest 3 short reply options for this message:
```

```
"${message.author.name}: ${message.content}"

Return JSON: { replies: ["reply1", "reply2", "reply3"] }
Make replies natural, varied (one agreeing, one asking follow
-up, one providing info).`,
        { systemPrompt: "Generate natural reply suggestion
s.", temperature: 0.7 },
      );
      return result;
    }),

  // AI compose in messaging
  compose: protectedProcedure
    .input(z.object({
      action: z.enum(["draft_reply", "make_professional", "tr
anslate"]),
      inputText: z.string().optional(),
      threadContext: z.string().optional(),
      language: z.string().optional(),
    }))
    .mutation(async ({ input }) => {
      const prompts: Record<string, string> = {
        draft_reply: `Draft a reply to this conversation:\n
${input.threadContext}\n\nDraft a helpful, professional respo
nse.`,
        make_professional: `Rewrite this message professional
ly:\n"${input.inputText}"`,
        translate: `Translate to ${input.language || "Englis
h"}:\n"${input.inputText}"`,
      };

      const result = await aiService.generateText(prompts[inp
ut.action], {
        systemPrompt: "Write clear, professional messages.",
maxTokens: 500, temperature: 0.5,
      });
```

```
      return { text: result.text };
    }),

  // Extract decisions from channel
  extractDecisions: protectedProcedure
    .input(z.object({ channelId: z.string().uuid(), dateRang
e: z.object({ start: z.string(), end: z.string() }) }))
    .mutation(async ({ input }) => {
      const messages = await prisma.message.findMany({
        where: { channelId: input.channelId, createdAt: { gt
e: new Date(input.dateRange.start), lte: new Date(input.dateR
ange.end) } },
        include: { author: { select: { name: true } } },
        orderBy: { createdAt: "asc" },
        take: 200,
      });

      const result = await aiService.generateJSON<{ decision
s: Array<{ decision: string; decidedBy: string; context: stri
ng; date: string }> }>(
        `Extract all decisions made in these messages:\n${mes
sages.map(m => `[${m.createdAt.toISOString()}] ${m.author.nam
e}: ${m.content}`).join("\n")}

Return: { decisions: [{decision, decidedBy, context, date}] }
`,
        { systemPrompt: "Extract team decisions from conversa
tion history.", temperature: 0.2 },
      );
      return result;
    }),
});
```

FEATURE 13: AI EMAIL ASSISTANT

tRPC Router: `src/server/routers/email.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const emailRouter = router({
  // AI compose email
  compose: protectedProcedure
    .input(z.object({
      action: z.enum(["compose", "reply", "follow_up"]),
      subject: z.string().optional(),
      to: z.string().optional(),
      threadContext: z.string().optional(),
      instructions: z.string().optional(),
      tone: z.enum(["professional", "casual", "friendly", "fo
rmal"]).default("professional"),
    }))
    .mutation(async ({ input }) => {
      const prompts: Record<string, string> = {
        compose: `Write an email:\nTo: ${input.to}\nSubject:
${input.subject}\nInstructions: ${input.instructions}\nTone:
${input.tone}`,
        reply: `Write a reply to this email thread:\n${input.
threadContext}\n\nInstructions: ${input.instructions || "Writ
e a professional reply."}\nTone: ${input.tone}`,
        follow_up: `Write a follow-up email for this threa
d:\n${input.threadContext}\n\nInstructions: ${input.instructi
ons || "Politely follow up."}\nTone: ${input.tone}`,
      };

      const result = await aiService.generateText(prompts[inp
ut.action], {
        systemPrompt: "Write professional emails. Include sub
ject line if composing new email.",
```

```
        maxTokens: 1000, temperature: 0.5,
      });
      return { text: result.text };
    }),

  // Triage emails by priority
  triageInbox: protectedProcedure
    .input(z.object({ emailAccountId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const threads = await prisma.emailThread.findMany({
        where: { emailAccountId: input.emailAccountId, aiPrio
rity: null },
        take: 50,
        orderBy: { lastMessageAt: "desc" },
      });

      for (const thread of threads) {
        const result = await aiService.generateJSON<{ priorit
y: string; labels: string[] }>(
          `Classify this email:
Subject: ${thread.subject}
Preview: ${thread.latestSnippet}
From: ${JSON.stringify(thread.participants)}

Return: { priority: 'urgent'|'important'|'fyi'|'low', labels:
['action_required'|'waiting'|'informational'|'scheduled'] }`,
          { systemPrompt: "Triage emails by priority.", tempe
rature: 0.2 },
        );

        await prisma.emailThread.update({
          where: { id: thread.id },
          data: { aiPriority: result.priority, aiLabels: resu
lt.labels },
        });
      }
```

```
      return { triaged: threads.length };
    }),

  // Summarize email thread
  summarize: protectedProcedure
    .input(z.object({ threadId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const thread = await prisma.emailThread.findUnique({ wh
ere: { id: input.threadId } });
      if (!thread) throw new Error("Thread not found");

      const result = await aiService.generateText(
        `Summarize this email thread:\nSubject: ${thread.subj
ect}\nContent: ${thread.latestSnippet}`,
        { systemPrompt: "Summarize email threads concisely.",
maxTokens: 300, temperature: 0.3 },
      );

      await prisma.emailThread.update({
        where: { id: input.threadId },
        data: { aiSummary: result.text },
      });

      return { summary: result.text };
    }),
});
```

VERIFICATION — SPRINT 7

```
□ Click "Summarize" on a long thread → 3-5 sentence summary a
ppears
□ Click "Catch me up" in channel → summary since last visit s
hown
□ Reply suggestions appear as quick-reply pills below message
```

```
s
□ AI compose in message composer → draft generated
□ Extract decisions from channel → list of decisions with wh
o/when
□ AI compose email → professional email generated
□ Inbox triage → emails categorized as urgent/important/fyi/l
ow
□ Email thread summarization works
```

Continue to Sprints 8-10 in the next file.